

Unity Fake Null이란?

요약

Unity의 `UnityEngine.Object`는 `==` 연산자와 `bool` 연산자를 오버라이드하여, **C# 참조는 살아있지만 네이티브 객체는 파괴된 상태를 `null`처럼 취급한다**. 이를 "Fake Null"이라 부른다. C#의 `null` 체크 구문 중 Unity 오버라이드를 거치지 않는 것들은 Fake Null을 감지하지 못한다.

1. 배경

Unity는 C# 래퍼 객체와 C++ 네이티브 객체의 이중 구조로 동작한다.

[C# Managed]		[C++ Native]
MonoBehaviour	→	Native GameObject
(래퍼)		(실제 엔진 객체)

`Destroy(obj)`를 호출하면:

1. **C++ 네이티브 객체**는 프레임 끝에서 파괴된다.
2. **C# 래퍼 객체**는 GC가 수거할 때까지 메모리에 남아있다.

이 사이 기간에 C# 참조는 존재하지만 네이티브 객체는 없는 상태, 즉 Fake Null이 된다.

2. Unity의 연산자 오버라이드

출처: [UnityCsReference - UnityEngineObject.cs](#)

```
// == 연산자: 네이티브 객체 유효성까지 확인
public static bool operator ==(Object x, Object y) => CompareBaseObjects(x, y);

// bool 연산자: 네이티브 객체 유효성까지 확인
public static implicit operator bool(Object exists) => !CompareBaseObjects(exists,
null);
```

`CompareBaseObjects`는 내부적으로 C# 참조가 `null`인지뿐만 아니라, **네이티브 포인터(`m_CachedPtr`)가 유효한지까지 확인한다**. 따라서 Destroy된 객체에 대해 `true(== null)`를 반환한다.

3. Fake Null 감지 여부

구문	Fake Null 감지	이유
<code>obj == null</code>	○	Unity 오버라이드 <code>==</code> 사용

구문	Fake Null 감지	이유
<code>obj != null</code>	O	Unity 오버라이드 <code>!=</code> 사용
<code>!obj</code>	O	Unity 오버라이드 <code>implicit bool</code> 사용
<code>if (obj)</code>	O	Unity 오버라이드 <code>implicit bool</code> 사용
<code>obj?.Method()</code>	X	C# 컴파일러가 <code>ReferenceEquals(obj, null)</code> 기반으로 처리
<code>obj ?? fallback</code>	X	C# 컴파일러가 참조 null만 확인
<code>obj is null</code>	X	패턴 매칭은 <code>ReferenceEquals</code> 사용
<code>obj is not null</code>	X	패턴 매칭은 <code>ReferenceEquals</code> 사용
<code>ReferenceEquals(obj, null)</code>	X	CLR 수준의 참조 비교만 수행

4. 위험한 사용 예시

Null 조건 연산자 (`?.`)

```
Destroy(gameObject);

// 다음 프레임 이후
gameObject?.SetActive(true); // 실행됨! Fake Null을 감지 못함
// MissingReferenceException 발생
```

Null 병합 연산자 (`??`)

```
Destroy(_target);

// 다음 프레임 이후
var obj = _target ?? fallbackObject; // _target이 선택됨!
// Fake Null인 _target을 유효한 객체로 취급
```

패턴 매칭 (`is null`)

```
Destroy(_target);

// 다음 프레임 이후
if (_target is not null)
{
    _target.SetActive(true); // 실행됨!
```

```
// MissingReferenceException 발생
}
```

5. 안전한 패턴

```
// Unity 오버라이드를 거치는 안전한 null 체크
if (obj != null)
    obj.SetActive(true);

if (obj)
    obj.SetActive(true);

// 삼항 연산자도 == 를 사용하면 안전
var result = (obj != null) ? obj : fallback;
```

6. 결론

항목	내용
Fake Null이란	C# 래퍼는 존재하나 네이티브 객체가 파괴된 상태
감지 가능	==, !=, if(obj), !obj (Unity 연산자 오버라이드 경유)
감지 불가	?., ??, is null, ReferenceEquals (C# 언어 수준 처리)
권장	UnityEngine.Object 계열은 항상 ==/!= 또는 bool 캐스팅으로 null 체크

C# 최신 구문(?., ??, is null)은 순수 C# 객체에는 적합하지만, UnityEngine.Object에는 Fake Null 문제로 인해 사용을 피해야 한다.